

Divyatra

Divyatra is a production-oriented spiritual experience platform built with FastAPI, MongoDB, Redis, Celery, Bootstrap 5, Chart.js, Razorpay, and WebSockets.

Features

- JWT auth with roles: `user`, `admin`, `staff`
- OTP verification workflow
- Online seva and darshan booking
- Razorpay order creation and backend payment verification
- Counter booking and token generation for walk-ins
- Real-time queue screen using WebSockets
- QR-enabled PDF ticket download
- Live darshan streaming via YouTube embed
- Media gallery, videos, announcements, and admin CMS endpoints
- Notification templates with email/SMS/WhatsApp-ready workflow
- Admin analytics for bookings, revenue, peak slots, and seva popularity
- Mobile-ready REST APIs with JSON responses
- Docker and Nginx deployment scaffolding

Project Structure

```
```text
app/
 core/ # config, db, auth, rate limiting, celery, websockets
 routers/ # API and page routes
 schemas/ # request/response schemas
 services/ # payment, notifications, analytics
 utils/ # QR and template helpers
templates/ # Bootstrap pages
static/ # CSS and frontend JS
deploy/ # Nginx config
```
```

Environment Variables

Copy `.env.example` to `.env` and configure:

```
```env
APP_NAME=Divyatra
APP_ENV=development
SECRET_KEY=change-me
ACCESS_TOKEN_EXPIRE_MINUTES=1440
MONGODB_URL=mongodb://localhost:27017
MONGODB_DB=divyatra
REDIS_URL=redis://localhost:6379/0
MAIL_USERNAME=
MAIL_PASSWORD=
MAIL_FROM=
MAIL_PORT=587
MAIL_SERVER=smtp.gmail.com
MAIL_STARTTLS=true
MAIL_SSL_TLS=false
TWILIO_ACCOUNT_SID=
TWILIO_AUTH_TOKEN=
TWILIO_PHONE_NUMBER=
```
```

```
RAZORPAY_KEY_ID=
RAZORPAY_KEY_SECRET=
RAZORPAY_WEBHOOK_SECRET=
YOUTUBE_LIVE_URL=https://www.youtube.com/embed/live_stream?channel=YOUR_C
HANNEL_ID
BASE_URL=http://localhost:8000
```
```

## ## Setup Steps

1. Create and activate a Python virtual environment.
2. Install dependencies with `pip install -r requirements.txt`.
3. Start MongoDB and Redis locally, or use Docker Compose.
4. Copy `.env.example` to `.env` and fill in secrets and provider credentials.
5. Launch the app and worker processes.

## ## Run Instructions

### ### Local

Start the API server:

```
```bash
uvicorn app.main:app --reload
```
```

Start the Celery worker:

```
```bash
celery -A app.core.celery_app.celery_app worker --loglevel=info
```
```

Open the application at `http://localhost:8000`.

### ### Docker

```
```bash
docker compose up --build
```
```

This starts FastAPI, Celery, MongoDB, Redis, and Nginx.

## ## Default Roles

The app supports `user`, `admin`, and `staff`.

To promote users after registration, update the `role` field in MongoDB:

```
```javascript
db.users.updateOne({ email: "admin@example.com" }, { $set: { role:
"admin", otp_verified: true } })
db.users.updateOne({ email: "staff@example.com" }, { $set: { role:
"staff", otp_verified: true } })
```
```

## ## API Coverage

- ``/api/auth``: register, verify OTP, login
- ``/api/services``: list, create, update services
- ``/api/bookings``: create booking, verify payment, history, ticket, counter booking
- ``/api/queue``: list queue, advance queue, websocket updates
- ``/api/media``: gallery, videos, announcements, stream, media CMS
- ``/api/notifications``: templates, immediate send, reminder scheduling
- ``/api/analytics/dashboard``: reporting data for charts
- ``/api/admin``: overview, users, bookings
- ``/api/staff``: dashboard and counter booking views

## ## Security Notes

- JWT-based auth
- Password hashing with ``passlib``
- Role guards for admin and staff endpoints
- Redis-backed rate limiting on login and register
- Input validation through Pydantic
- Secure headers middleware
- HTTPS redirect toggle in production mode

## ## Notes

- Twilio and email providers are integration-ready; if credentials are absent, the notification flow falls back to safe mock/configured statuses.
- Razorpay can run in mock verification mode during development if live keys are not configured.
- To scale backend instances behind Nginx, add more FastAPI containers and extend the upstream block.